



LarvaLens

Backend Schema

Data, Auth and Security Architecture

Complete PostgreSQL/Supabase tables, constraints, indexes, role model, Row Level Security, profile trigger, storage paths, Realtime behavior and transaction rules.

Prepared for	Mohit S. and the LarvaLens team
Build format	Vibe-coding source-of-truth document 05
Version	1.0 - 26 August 2026



Vibe Document 05 - Backend Schema and Auth Architecture

Document ID: 05_BACKEND_SCHEMA.md Status: Source of truth

10.1 Roles

Role	Permissions
field_worker	Create scans; read own scans/tracks/reviews.
reviewer	Read review queue and evidence; create review decisions.
admin	Reviewer permissions plus model metadata administration.

Role authorization must come from a server-controlled profile/app claim. Do not trust editable client metadata.

10.2 Tables

Table	Purpose	Important fields
profiles	User identity/role extension	id, full_name, role, timestamps.
model_versions	Immutable artifact metadata	artifact kind, version label, SHA-256, stage epochs, metrics, active.
scans	One uploaded analysis job/result	owner, status, progress, paths, coordinates, counts, quality, risk, model hashes, error.
tracks	Track-level evidence	detector/larva/non-larva/motion scores, persistence, accepted, reason, trajectory.
reviews	Human decision separate from model evidence	reviewer, decision, notes, timestamp.
scan_events	Append-only processing audit	stage, event type, payload, timestamp.

10.3 Migration SQL - create as supabase/migrations/001_initial.sql

sql

```
create extension if not exists pgcrypto;

create type public.user_role as enum ('field_worker', 'reviewer', 'admin');
create type public.scan_status as enum (
  'queued', 'validating', 'detecting', 'verifying', 'tracking',
  'completed', 'retake_required', 'failed'
);
create type public.risk_level as enum ('none_observed', 'low', 'medium', 'high');
create type public.video_quality as enum ('good', 'usable', 'poor');
create type public.review_decision as enum ('confirmed', 'rejected', 'inconclusive');

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null check (char_length(full_name) between 1 and 120),
  role public.user_role not null default 'field_worker',
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

create table public.model_versions (
  id uuid primary key default gen_random_uuid(),
  artifact_kind text not null check (artifact_kind in ('detector', 'verifier', 'species')),
  version_label text not null,
  filename text not null,
  sha256 char(64) not null,
  cumulative_epoch_label integer,
  class_names jsonb not null default '[]'::jsonb,
  metrics jsonb not null default '{}'::jsonb,
  active boolean not null default false,
  created_at timestampz not null default now(),
```

```

    unique (artifact_kind, sha256)
);

create unique index one_active_model_per_kind
  on public.model_versions(artifact_kind) where active;

create table public.scans (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null references public.profiles(id) on delete cascade,
  status public.scan_status not null default 'queued',
  progress_percent smallint not null default 0 check (progress_percent between 0 and 100),
  current_stage text,
  source_video_path text not null,
  evidence_video_path text,
  source_mime_type text not null,
  source_size_bytes bigint not null check (source_size_bytes > 0),
  duration_seconds numeric(8,3),
  latitude numeric(9,6) check (latitude between -90 and 90),
  longitude numeric(9,6) check (longitude between -180 and 180),
  location_accuracy_m numeric(9,2),
  probable_larvae_count integer check (probable_larvae_count >= 0),
  rejected_tracks integer check (rejected_tracks >= 0),
  overall_confidence numeric(5,4) check (overall_confidence between 0 and 1),
  risk_level public.risk_level,
  video_quality public.video_quality,
  quality_reasons jsonb not null default '[]'::jsonb,
  model_versions jsonb not null default '{}'::jsonb,
  review_status text not null default 'pending' check (review_status in ('pending','reviewed','not_required')),
  error_code text,
  error_message text,
  idempotency_key text,
  created_at timestamptz not null default now(),
  started_at timestamptz,
  completed_at timestamptz,
  updated_at timestamptz not null default now(),
  unique(owner_id, idempotency_key)
);

create index scans_owner_created_idx on public.scans(owner_id, created_at desc);
create index scans_status_created_idx on public.scans(status, created_at);
create index scans_geo_idx on public.scans(latitude, longitude)
  where latitude is not null and longitude is not null;

create table public.tracks (
  id uuid primary key default gen_random_uuid(),
  scan_id uuid not null references public.scans(id) on delete cascade,
  track_number integer not null,
  detector_confidence numeric(5,4) not null check (detector_confidence between 0 and 1),
  larva_probability numeric(5,4) not null check (larva_probability between 0 and 1),
  non_larva_probability numeric(5,4) not null check (non_larva_probability between 0 and 1),
  motion_score numeric(9,5) not null check (motion_score >= 0),
  fused_confidence numeric(5,4) not null check (fused_confidence between 0 and 1),
  persistence_frames integer not null check (persistence_frames > 0),
  accepted boolean not null,
  reject_reason text,
  trajectory jsonb not null default '[]'::jsonb,
  evidence_frame_path text,
  created_at timestamptz not null default now(),
  unique(scan_id, track_number)
);

create index tracks_scan_idx on public.tracks(scan_id, accepted);

create table public.reviews (
  id uuid primary key default gen_random_uuid(),
  scan_id uuid not null references public.scans(id) on delete cascade,
  reviewer_id uuid not null references public.profiles(id),
  decision public.review_decision not null,
  notes text check (char_length(notes) <= 2000),
  created_at timestamptz not null default now(),
  unique(scan_id, reviewer_id)
);

create table public.scan_events (
  id bigint generated always as identity primary key,
  scan_id uuid not null references public.scans(id) on delete cascade,
  stage text not null,
  event_type text not null,
  payload jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);

create index scan_events_scan_idx on public.scan_events(scan_id, created_at);

```

10.4 Authorization helper and RLS

sal

```

create or replace function public.has_review_role()
returns boolean
language sql
stable
security definer
set search_path = public
as $$

```

```

select exists (
  select 1 from public.profiles
  where id = auth.uid() and role in ('reviewer', 'admin')
);
$$;

revoke all on function public.has_review_role() from public;
grant execute on function public.has_review_role() to authenticated;

alter table public.profiles enable row level security;
alter table public.model_versions enable row level security;
alter table public.scans enable row level security;
alter table public.tracks enable row level security;
alter table public.reviews enable row level security;
alter table public.scan_events enable row level security;

create policy profiles_read_own on public.profiles
for select to authenticated
using (auth.uid() is not null and id = auth.uid());

create policy profiles_update_own on public.profiles
for update to authenticated
using (auth.uid() is not null and id = auth.uid())
with check (auth.uid() is not null and id = auth.uid());

create policy model_versions_read on public.model_versions
for select to authenticated using (true);

create policy scans_insert_own on public.scans
for insert to authenticated
with check (auth.uid() is not null and owner_id = auth.uid());

create policy scans_read_own_or_reviewer on public.scans
for select to authenticated
using (owner_id = auth.uid() or public.has_review_role());

create policy tracks_read_own_or_reviewer on public.tracks
for select to authenticated
using (exists (
  select 1 from public.scans s
  where s.id = tracks.scan_id
  and (s.owner_id = auth.uid() or public.has_review_role())
));

create policy reviews_read_related on public.reviews
for select to authenticated
using (reviewer_id = auth.uid() or exists (
  select 1 from public.scans s
  where s.id = reviews.scan_id and s.owner_id = auth.uid()
) or public.has_review_role());

create policy reviews_insert_reviewer on public.reviews
for insert to authenticated
with check (reviewer_id = auth.uid() and public.has_review_role());

create policy scan_events_read_related on public.scan_events
for select to authenticated
using (exists (
  select 1 from public.scans s
  where s.id = scan_events.scan_id
  and (s.owner_id = auth.uid() or public.has_review_role())
));

```

The backend service role performs processing updates/inserts after verifying the caller and scan ownership. Client policies intentionally do not allow users to rewrite model results, tracks, events or model versions.

10.5 Profile trigger

sal

```

create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
  insert into public.profiles(id, full_name, role)
  values (
    new.id,
    coalesce(nullif(new.raw_user_meta_data->>'full_name', ''), split_part(new.email, '@', 1)),
    'field_worker'
  );
  return new;
end;
$$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute procedure public.handle_new_user();

```

Never let signup metadata choose `reviewer` or `admin`.

10.6 Storage paths and policies

Private bucket paths:

text

```
scan-videos/{owner_uuid}/{scan_uuid}/source.ext
scan-evidence/{owner_uuid}/{scan_uuid}/annotated.mp4
scan-evidence/{owner_uuid}/{scan_uuid}/tracks/{track_number}.jpg
```

Clients may upload only through the API for the MVP. The API generates paths, uploads with the service role and checks row ownership before issuing short-lived signed URLs. This avoids client-controlled paths and keeps private media out of public buckets.

10.7 Realtime

Add `public.scans` to the Realtime publication. The web subscribes with a primary-key filter for one authorized `scan_id`, listens for UPDATE, validates the payload shape and refetches the row on completion or reconnect. RLS still applies to Postgres Changes.

10.8 Transaction rule for completion

The backend must not mark a scan `completed` before track inserts and result fields succeed. Use a database function/RPC or transaction that inserts tracks, updates result fields, appends a completion event and sets `completed_at` atomically. On failure, set `failed` with a safe error code and retain no partial result as final.